This works well for many cases, but it has the distinct disadvantage that it requires the calling code to construct an instance of the stack's value type prior to the call, in order to pass this in as the target. For some types this is impractical, because constructing an instance is expensive in terms of time or resources. For other types this isn't always possible, because the constructors require parameters that aren't necessarily available at this point in the code. Finally, it requires that the stored type is assignable. This is an important restriction: many user-defined types do not support assignment, though they may support move construction or even copy construction (and thus allow return by value).

Option 2: Require a No-Throw Copy Constructor or Move Constructor

There's only an exception safety problem with a value-returning `pop()` if the return by value can throw an exception. Many types have copy constructors that don't throw exceptions, and with the new rvalue-reference support in the C++ Standard (see appendix A, section A.1), many more types will have a move constructor that doesn't throw exceptions, even if their copy constructor does. One valid option is to restrict the use of your thread-safe stack to those types that can safely be returned by value without throwing an exception.

Although this is safe, it's not ideal. Even though you can detect at compile time the existence of a copy or move constructor that doesn't throw an exception using the `std::is_nothrow_copy_constructible` and `std::is_nothrow_move_constructible` type traits, it's quite limiting. There are many more user-defined types with copy constructors that can throw and don't have move constructors than there are types with copy and/or move constructors that can't throw (although this might change as people get used to the rvalue-reference support in C++11). It would be unfortunate if such types couldn't be stored in your thread-safe stack.

Option 3: Return a Pointer to the Popped Item

The third option is to return a pointer to the popped item rather than return the item by value. The advantage here is that pointers can be freely copied without throwing an exception, so you've avoided Cargill's exception